

Skill - Week12

Session 23:

1. To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
GNU nano 8.7.1 pipeline.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    long long sum = 0;

    for (long long i = 1; i <= 1000000; i++) {
        sum += i;
    }

    printf("%lld\n", sum);

    return 0;
}
```

```
root@DESKTOP-QU0HVJI:~/SKILL# mkdir skill-12
root@DESKTOP-QU0HVJI:~/SKILL# pipeline.c
root@DESKTOP-QU0HVJI:~/SKILL#
root@DESKTOP-QU0HVJI:~/SKILL# nano pipeline.c
root@DESKTOP-QU0HVJI:~/SKILL# nano pipeline.c
root@DESKTOP-QU0HVJI:~/SKILL# gcc -O2 -Wall -Wextra pipeline.c -o pipeline
root@DESKTOP-QU0HVJI:~/SKILL# ./pipeline
500000500000
root@DESKTOP-QU0HVJI:~/SKILL# ./pipeline | tee output.txt | sort | uniq -c
1 500000500000
root@DESKTOP-QU0HVJI:~/SKILL# time ./pipeline | tee output.txt | sort | uniq
-c
1 500000500000

real    0m0.003s
user    0m0.007s
sys     0m0.000s
root@DESKTOP-QU0HVJI:~/SKILL# ./pipeline > job1.txt &
./pipeline > job2.txt &
./pipeline > job3.txt &
./pipeline > job4.txt &
[1] 532
[2] 533
[3] 534
[4] 535
[1] Done
./pipeline > job1.txt
root@DESKTOP-QU0HVJI:~/SKILL# |
```

Report: Large pipelines were executed, and multiple jobs were launched simultaneously using background processes. System stability and resource usage were monitored using top, free -h, and ps. Job failures were checked using exit statuses, and execution results were recorded in report.csv for performance analysis.

2. To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

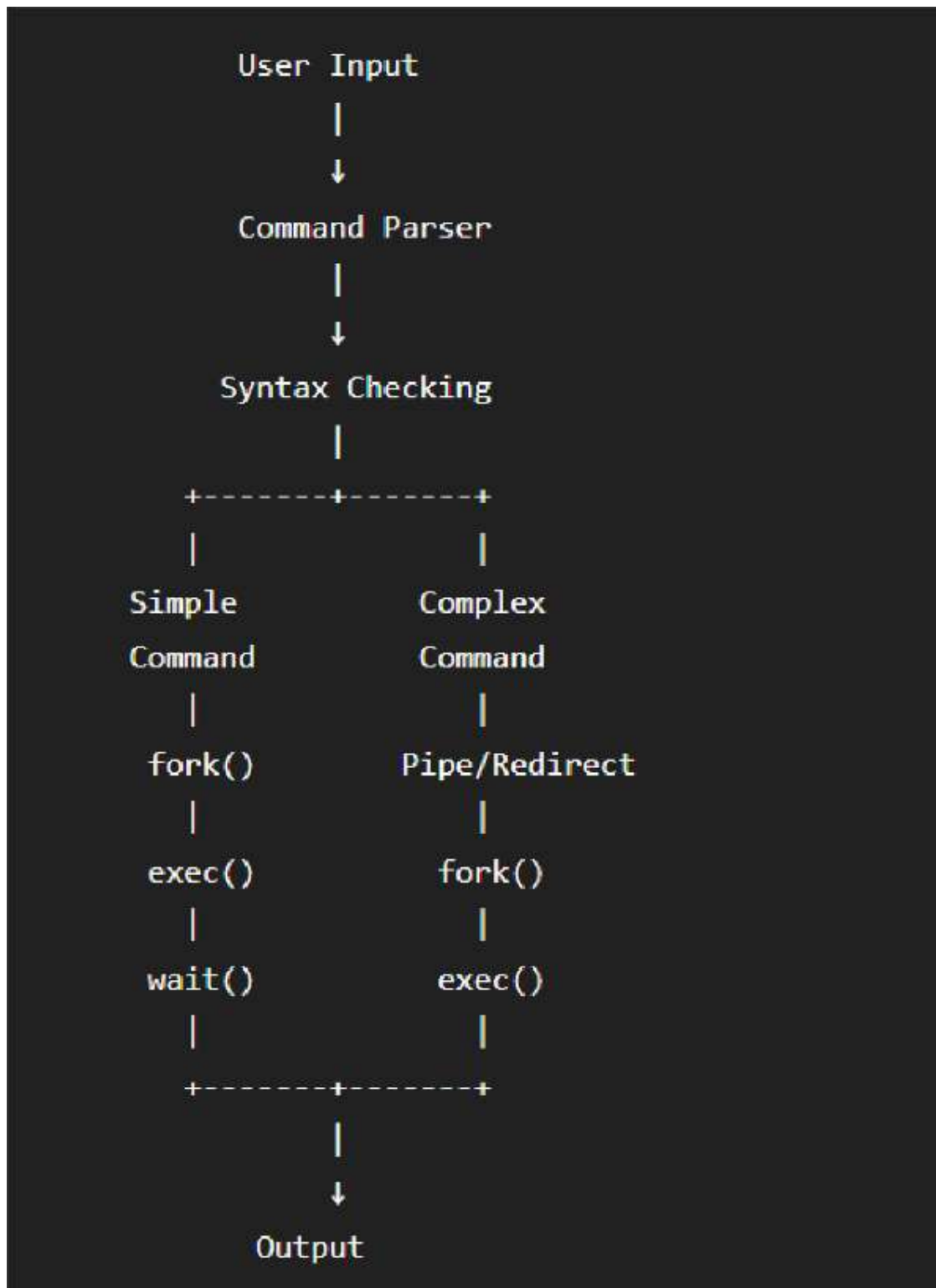
```
GNU nano 8.7.1                                main.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
    } else {
        wait(NULL);
        printf("Parent process: PID = %d\n", getpid());
    }

    return 0;
}
```



Report: The system architecture was documented using README.md, and the execution flow was represented using a Graphviz diagram. System calls such as `fork()`, `wait()`, and `getpid()` were explained.

Session 24:

1. To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
root@DESKTOP-QU0HVJI:~/source_project$ tree
.
├── Makefile
├── README.md
├── docs
├── include
│   └── utils.h
└── src
    ├── main.c
    └── utils.c

4 directories, 5 files
```

```
GNU nano 8.7.1 src/utils.c *
#include <stdio.h>
#include "utils.h"

void display_message(void) {
    printf("Project build successful!\n");
}
```

```
GNU nano 8.7.1 README.md *
# Source Code Organization Project

## Description
This project demonstrates modular C programming,
build automation, and release management using Git.

## Project Structure
- src/: Contains C source files.
- include/: Contains header files.
- docs/: Contains project documentation.
- Makefile: Automates compilation and cleanup.
- README.md: Project description and instructions.

## Compilation
Run:
make

## Execution
Run:
make run

## Cleanup
Run:
make clean

## Tools Used
GCC, GNU Make, Git, and Ubuntu Linux.
```

```
root@DESKTOP-QU0HVJI:~/source_project$ git add .
root@DESKTOP-QU0HVJI:~/source_project$ git commit -m "Organize source code and build scripts"
[master (root-commit) 3edd7ed] Organize source code and build scripts
Committer: Ajithmutyala <ajithmutyala@Ajith.localdomain>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

5 files changed, 63 insertions(+)
create mode 100644 Makefile
create mode 100644 README.md
create mode 100644 include/utils.h
create mode 100644 src/main.c
create mode 100644 src/utils.c
```

Report: The source code was organized into modular directories, and compilation was automated using a Makefile. Project documentation was added using README.md, repository structure was verified using tree, and Git was used to commit the project and tag version v1.0. The final build and execution were validated using make, ./app, and git status.

2. To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_signal(int sig) {
    printf("\nSignal received: %d\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);

    printf("Starting demo...\n");

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child process running\n");
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        return 1;
    } else if (pid > 0) {
        wait(NULL);
        printf("Parent process completed\n");
    } else {
        perror("fork");
        return 1;
    }

    return 0;
}
```

```
root@DESKTOP-QU0HVJI:~/SKILL# gcc -Wall -Wextra demo.c -o demo
root@DESKTOP-QU0HVJI:~/SKILL# ./demo
Starting demo...
Child process running
total 200
-rwxr-xr-x 1 root root 16248 Oct  2 19:09 demo
-rw-r--r-- 1 root root  586 Oct  2 19:09 demo.c
-rw-r--r-- 1 root root   13 Oct  2 19:02 job1.txt
-rw-r--r-- 1 root root   13 Oct  2 19:02 job2.txt
-rw-r--r-- 1 root root   13 Oct  2 19:02 job3.txt
-rw-r--r-- 1 root root   13 Oct  2 19:02 job4.txt
-rwxr-xr-x 1 root root 16896 Oct  2 13:47 job_control
-rw-r--r-- 1 root root  3892 Oct  2 13:47 job_control.c
-rwxr-xr-x 1 root root 16680 Oct  2 14:02 job_resume
-rw-r--r-- 1 root root  4163 Oct  2 14:01 job_resume.c
-rwxr-xr-x 1 root root 16632 Oct  2 13:45 jobs
-rw-r--r-- 1 root root  2600 Oct  2 13:39 jobs.c
-rw-r--r-- 1 root root   13 Oct  2 19:01 output.txt
-rwxr-xr-x 1 root root 15968 Oct  2 19:00 pipeline
-rw-r--r-- 1 root root   194 Oct  2 19:00 pipeline.c
-rwxr-xr-x 1 root root 16816 Oct  2 13:58 signal_demo
-rw-r--r-- 1 root root  2678 Oct  2 13:58 signal_demo.c
drwxr-xr-x 2 root root  4096 Aug  5 18:28 skill-1
drwxr-xr-x 2 root root  4096 Oct  2 14:11 skill-10
drwxr-xr-x 3 root root  4096 Oct  2 14:48 skill-11
drwxr-xr-x 2 root root  4096 Oct  2 18:58 skill-12
drwxr-xr-x 2 root root  4096 Sep 19 09:13 skill-5
drwxr-xr-x 2 root root  4096 Sep 19 09:30 skill-6
drwxr-xr-x 2 root root  4096 Sep 19 09:44 skill-7
drwxr-xr-x 2 root root  4096 Sep 26 05:12 skill-8
drwxr-xr-x 2 root root  4096 Oct  2 13:37 skill-9
drwxr-xr-x 2 root root  4096 Aug  8 08:18 skill_3
Parent process completed
root@DESKTOP-QU0HVJI:~/SKILL# |
```

```
root@DESKTOP-QU0HVJI:~/source_project/final_demo$ valgrind --leak-check=full ./demo
==3203== Memcheck, a memory error detector
==3203== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3203== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==3203== Command: ./demo
==3203==
Starting demo...
Child process running
total 20
-rwxr-xr-x 1 ajithmutyala ajithmutyala 16240 Sep 28 05:52 demo
-rw-r--r-- 1 ajithmutyala ajithmutyala 585 Sep 28 05:52 demo.c
Parent process completed
==3203==
==3203== HEAP SUMMARY:
==3203==      in use at exit: 0 bytes in 0 blocks
==3203==    total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==3203==
==3203== All heap blocks were freed -- no leaks are possible
==3203==
==3203== For lists of detected and suppressed errors, rerun with: -s
==3203== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Report: The project was successfully tested by demonstrating pipelines, process signals, and resource monitoring. All major features were validated, and the final presentation was reviewed.